T3000 API & Protocol

Building Platform Overview · built Nov 13, 2026

Building Platform Overview

The T3000 Building Platform is the core engine that powers the building automation system. It handles communication between the web interface and physical devices, manages data flow, and executes control commands.

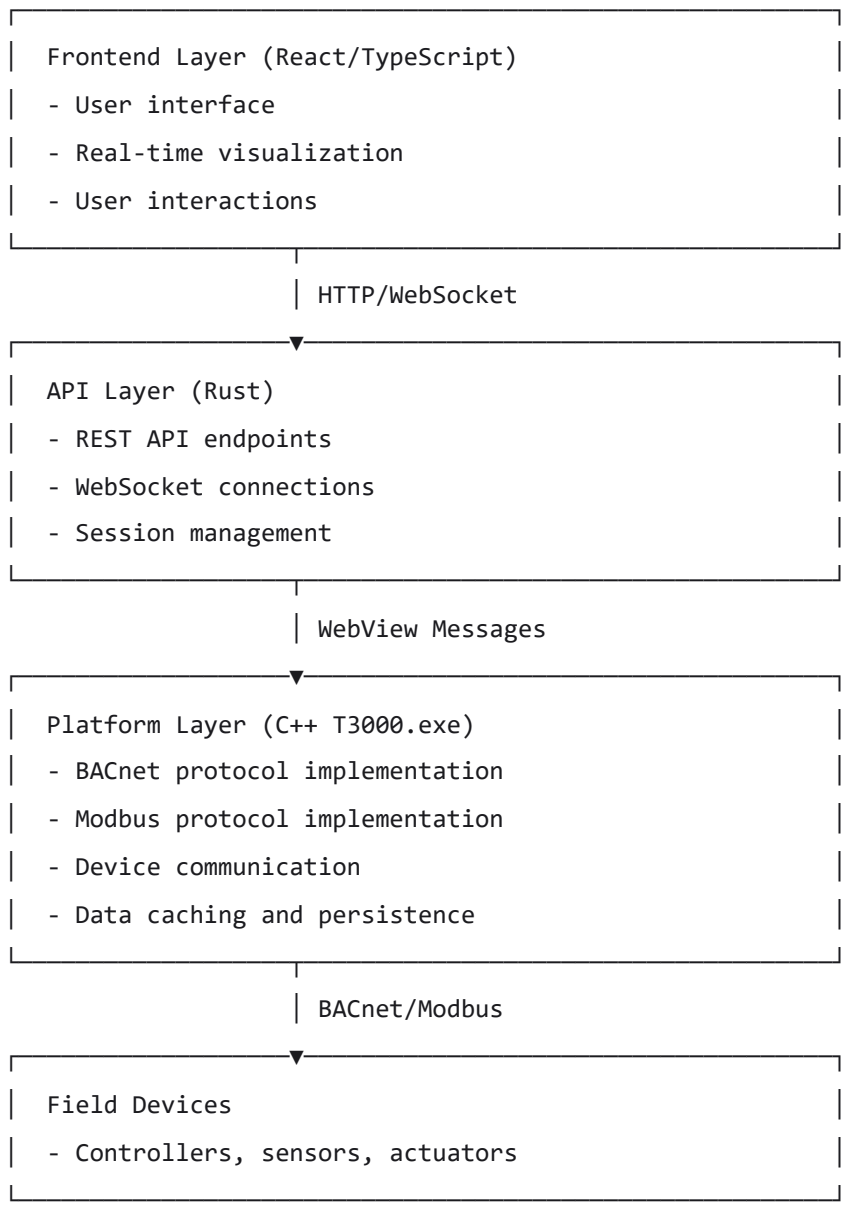
What You'll Find Here:

- System architecture and communication flow
- Control message reference
- BACnet protocol integration
- Data structures and models

Architecture

T3000 uses a three-layer architecture:

TEXT



Communication Flow

Request Flow (Frontend → Devices)

1. **User Action** - User clicks button or updates value in web UI
2. **API Request** - React app sends HTTP/WebSocket request to Rust API
3. **WebView Message** - Rust API creates JSON message and sends to C++ via WebView
4. **Message Handler** - C++ `HandleWebViewMsg()` processes message
5. **BACnet Command** - C++ sends BACnet read/write command to device
6. **Device Response** - Device responds with data
7. **JSON Response** - C++ formats response as JSON
8. **API Response** - Rust forwards to frontend

9. **UI Update** - React updates interface

Background Sync Flow

- **Trendlog Collection** - Automatic data logging every minute
- **Device Polling** - Periodic status checks
- **Cache Updates** - Persistent storage of device data
- **Event Notifications** - Real-time alarms and events

Control Messages

Control messages are the primary communication mechanism between the API layer and Platform layer. Each message has:

- **Message Type** - Enum identifier (e.g., GET_WEBVIEW_LIST)
- **Request Parameters** - JSON payload with required data
- **Response Format** - Structured JSON response
- **Error Handling** - Standardized error messages

Message Categories

Category	Purpose	Examples
Data Retrieval	Reading device data	GET_PANEL_DATA, GET_WEBVIEW_LIST
Data Updates	Writing to devices	UPDATE_WEBVIEW_LIST, UPDATE_ENTRY
Graphics	Graphics screen management	LOAD_GRAPHIC_ENTRY, SAVE_GRAPHIC_DATA
Discovery	Device and panel listing	GET_PANELS_LIST, GET_ENTRIES

View All Control Messages →

BACnet Integration

T3000 implements BACnet protocol for device communication:

Read Commands

- `READINPUT_T3000` - Read analog/digital inputs
- `READOUTPUT_T3000` - Read outputs
- `READVARIABLE_T3000` - Read variables
- `READPROGRAM_T3000` - Read programs
- `READCONTROLLER_T3000` - Read PID controllers

- `READMONITOR_T3000` - Read trendlog monitors

Write Commands

- `WRITEINPUT_T3000` - Write input configurations
- `WRITEOUTPUT_T3000` - Write output values
- `WRITEVARIABLE_T3000` - Write variable values
- `WRITEPROGRAM_T3000` - Write program code

Reading Strategy

Data is read in **chunks (groups)** to optimize network traffic:

```C++
// Example: Reading 64 inputs in groups of 8
for (int i = 0; i < 8 groups; i++)
{
    // Read 8 items per iteration
    ReadInputs(start: i*8, end: i*8+7)
}
```

Data Flow

Global Data Arrays

The platform maintains in-memory caches of device data:

```C++
vector<vector<Str_in_point>> g_Input_data;        // Inputs
vector<vector<Str_out_point>> g_Output_data;      // Outputs
vector<vector<Str_variable_point>> g_Variable_data; // Variables
vector<vector<Str_program_point>> g_Program_data; // Programs
vector<vector<Str_controller_point>> g_controller_data; // Controllers
vector<vector<Str_monitor_point>> g_monitor_data; // Monitors
```

First dimension: Panel ID (0-254) **Second dimension:** Point index (varies by type)

Temporary Buffers

During reads, data is first loaded into temporary buffers:

```
C++

Str_in_point s_Input_data[BAC_INPUT_ITEM_COUNT];
Str_program_point s_Program_data[BAC_PROGRAM_ITEM_COUNT];
```

Then copied to global arrays after successful read.

Data Structures

Common Point Structure Pattern

All BACnet point types follow similar structure:

```
C++

typedef struct {
    char description[DESCRIPTION_LENGTH];  // Full description
    char label[LABEL_LENGTH];              // Short label
    // ... type-specific fields ...
    uint8_t auto_manual;                   // Auto(0) or Manual(1)
    uint16_t value;                        // Current value
    uint8_t range;                         // Unit/range selection
} Str_xxx_point;
```

View Data Structure Reference →

File Organization

Platform Code Location:

TEXT

```
T3000-Source/
└── T3000/
    ├── BacnetWebView.cpp        - WebView message handler (main entry)
    ├── BacnetWebView.h          - Message type definitions
    ├── global_define.h          - Constants and definitions
    ├── global_variable.h        - Global data arrays
    ├── CM5/ud_str.h             - Data structure definitions
    ├── BacnetInput.cpp          - Input handling
    ├── BacnetOutput.cpp         - Output handling
    ├── BacnetVariable.cpp       - Variable handling
    ├── BacnetProgram.cpp        - Program handling
    ├── BacnetController.cpp     - Controller handling
    └── BacnetMonitor.cpp        - Monitor/trendlog handling
```

Performance Considerations

Caching Strategy

- **In-Memory Cache** - All device data cached in RAM for fast access
- **File Persistence** - Periodic writes to `DatabaseBak/` folder
- **Smart Refresh** - Only read changed data when possible

Network Optimization

- **Chunked Reads** - Large data sets split into manageable groups
- **Retry Logic** - Automatic retry on timeout (up to 4 attempts)
- **Delay Timing** - 200ms delay between commands to prevent flooding

Thread Safety

- **Blocking Reads** - `GetPrivateDataSaveSPBlocking()` ensures synchronous operations
- **Message Queue** - WebView messages queued and processed sequentially

Error Handling

Standard error response format:

JSON

```
{
    "action": "ERROR_RES",
    "error": {
        "code": -1,
        "message": "Read timeout",
        "details": "Failed to read inputs 0-7 from device 123456"
    }
}
```

Common error scenarios:

- **Device offline** - No response within timeout period
- **Invalid range** - Index out of bounds
- **Cache miss** - No cached data available
- **Permission denied** - Device access restricted

Next Steps

- Control Messages Reference →
- Message 17: GET_WEBVIEW_LIST →
- BACnet Commands →
- Data Structures →